

GPU image rotation and responsible HPC for AI

CUDA design and performance • Evidence, trade-offs and accountable use

Savin Vindiv De Alwis • 35221631

sdea0018@student.monash.edu

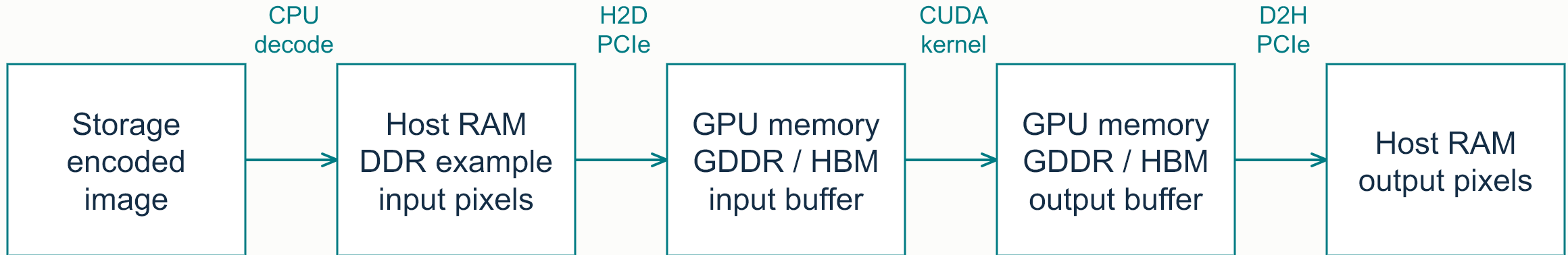
Shamle Thilaksiri • 35512075

wthi0003@student.monash.edu

Host–GPU image data flow

CPU: load/decode, allocate, launch, save

GPU: read input, rotate, write output



H2D: host → device. D2H: device → host. Separate buffers; DMA (direct memory access) engines transfer bytes over PCIe.

GPU memory bandwidth differs from PCIe bandwidth. Pinned RAM supports DMA; pageable RAM may stage. Complete output before saving or reuse.

The CPU orchestrates I/O and launches; the GPU rotates decoded pixels.

Rotation as independent output work

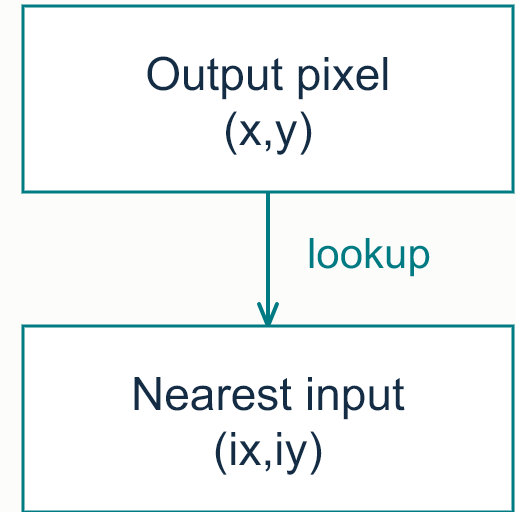
Inverse mapping in y-down image coordinates

```
sx = c*(x-cx) - s*(y-cy) + cx;  
sy = s*(x-cx) + c*(y-cy) + cy;  
ix = floor(sx + 0.5);  
iy = floor(sy + 0.5);
```

$c = \cos \theta$; $s = \sin \theta$; θ in radians; center = $((W-1)/2, (H-1)/2)$

Forward $[c \ s; -s \ c]$; inverse $[c \ -s; s \ c]$. Positive θ is visually counterclockwise.

Fixed RGB canvas can crop; black borders. Gathering avoids scatter holes; nearest-neighbour can alias.



One thread owns one output pixel: gather a source value, then write once.

CUDA maps pixels to scheduled threads

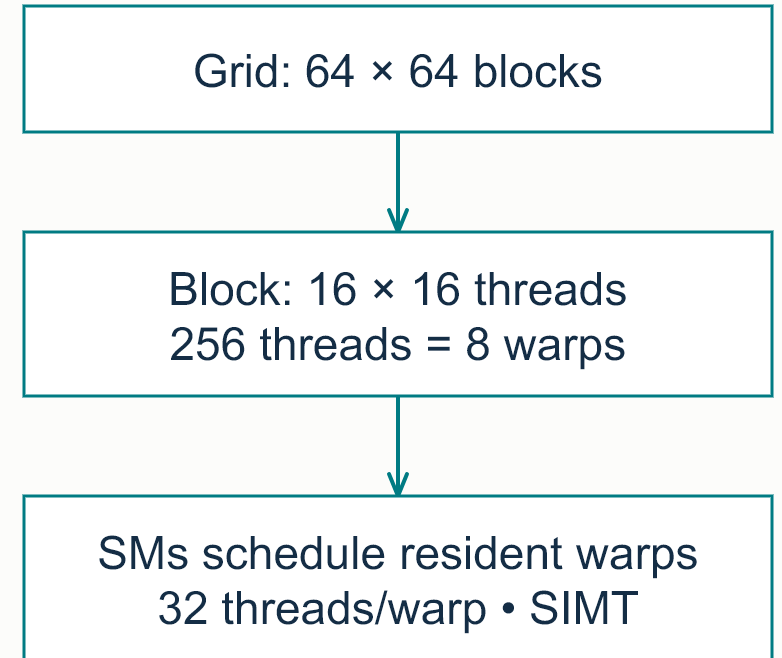
Host launch + device pixel indexing

```
dim3 block(16,16), grid(64,64);
rotate_rgb<<<grid,block>>>(
    d_in,d_out,1024,1024,c,s);
int x = blockIdx.x*blockDim.x + threadIdx.x;
int y = blockIdx.y*blockDim.y + threadIdx.y;
if (x >= W || y >= H) return;
```

Second kernel feature: clamp a brightness channel

```
int v = int(in[p]) + delta;
out[p] = v<0 ? 0 : (v>255 ? 255 : v);
```

After bounds checks: $p = 3 \cdot (\text{size_t}(y) \cdot W + x) + \text{ch}$; $\text{ch} \in [0, 2]$; $\text{delta} \in [-255, 255]$.
Separate RGB buffers. Uncompiled excerpts.



Residency hides latency; registers and divergence limit execution.

A logical grid exposes work; SMs execute resident warps with SIMT.

Throughput gains depend on the bottleneck

Why a GPU can be faster

Many execution units and high device-memory bandwidth process independent pixels concurrently.

Coalesced writes help; rotated gathers, registers and divergence can limit throughput.

Large/resident images amortize setup. Streams can overlap transfers/compute with separate pinned buffers and supporting hardware.

Compare completed equivalent work

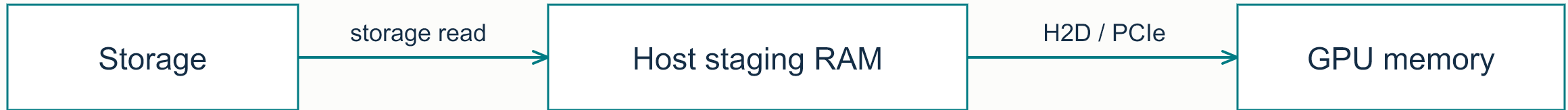
Speedup = $T_{CPU} / T_{GPU, total}$. Same rotation/sampling on one CPU thread; also test a multithreaded CPU.

Sequential time: setup + H2D + kernel + D2H. For overlapped batches, measure the completed critical path; wait before reusing buffers.

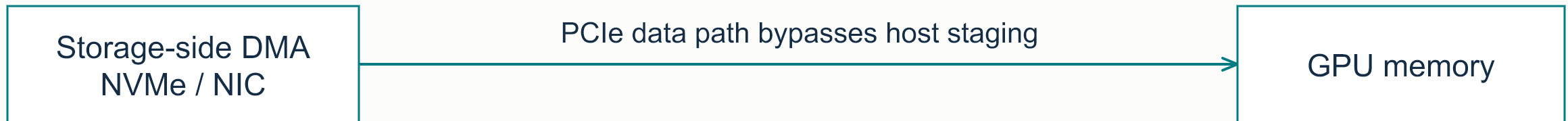
Amdahl: serial work bounds gains. No measured speedup; compile, verify and profile.

GPUDirect Storage and rotation

Conventional storage path



GDS direct path (supported configuration)



CPU issues cuFile I/O. Fallback may stage through RAM; GDS does not decode images or accelerate kernel arithmetic.

Use conventional copies for CPU-decoded / already-in-RAM pixels. Consider GDS for storage-bound GPU-ready batches only with supported GPU/filesystem/driver/topology and lower completed batch time.

GDS can reduce storage staging; it does not accelerate rotation arithmetic.

Scale useful work, then verify the outcome

Task 1: conditional performance

Independent inverse-gather pixels suit CUDA; locality, transfers and SM scheduling determine speedup.

GDS helps only a compatible storage-bound path.
Next: compile, compare CPU/GPU output and measure completed workloads.

Task 2: accountable scaling

Measure energy and useful output; protect data, evaluate group harms and allocate access transparently.

Next: test a real cluster job and review outcomes.
Sector projections and proposed policies are evidence limits, not measured results.

Detailed answers and supporting evidence

Task 1: implementation and analysis

Complete rotation and brightness examples, thread/grid calculations, host launch and stream completion.

Worked rotation, coordinate assumptions, launch-shape trade-offs, Amdahl bound, profiling and GPUDirect Storage decision.

Task 2: research and policy

Parallel training, energy/carbon calculations, environmental accounting and research limitations.

Risk owners, fairness example, allocation policy, framework roles, carbon-aware scheduling and cross-topic comparisons.

A 90-degree rotation example

Input

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

$\theta = 90^\circ$
 $c = 0, s = 1$
 center = (2,2)

Output: 90° counterclockwise

4	9	14	19	24
3	8	13	18	23
2	7	12	17	22
1	6	11	16	21
0	5	10	15	20

Output (0,1) → source (3,0) → value 3

The matrix values are illustrative pixel identifiers, not channel intensities.

CUDA launch and execution hierarchy

```
dim3 block(16,16);  
dim3 grid(64,64);  
rotate_rgb<<<grid,block>>>(  
    d_in,d_out,1024,1024,c,s);
```

```
x = blockIdx.x*blockDim.x + threadIdx.x;  
y = blockIdx.y*blockDim.y + threadIdx.y;  
if (x >= W || y >= H) return;
```

1024 × 1024 output

$64 \times 64 = 4,096$ blocks

$16 \times 16 = 256$ threads/block

1,048,576 logical threads

32 threads/warp → 8 warps/block

One highlighted warp within a 16-wide block



Non-multiple dimensions: ceil-divide the grid and keep the bounds guard.

A grid specifies logical work; finite SMs schedule its blocks and warps.

CUDA rotation kernel

```
__global__ void rotate_rgb(const unsigned char* in,
    unsigned char* out, int W, int H, float c, float s) {
    int x = blockIdx.x*blockDim.x + threadIdx.x;
    int y = blockIdx.y*blockDim.y + threadIdx.y;
    if (x >= W || y >= H) return;
    float cx = (W-1)*0.5f, cy = (H-1)*0.5f;
    int ix = int(floorf(c*(x-cx)-s*(y-cy)+cx+0.5f));
    int iy = int(floorf(s*(x-cx)+c*(y-cy)+cy+0.5f));
    size_t dst = 3*(size_t(y)*W + x);
    if (ix >= 0 && ix < W && iy >= 0 && iy < H) {
        size_t src = 3*(size_t(iy)*W + ix);
        for (int ch=0; ch<3; ++ch) out[dst+ch] = in[src+ch];
    } else {
        for (int ch=0; ch<3; ++ch) out[dst+ch] = 0;
    }
}
```

One thread owns one output pixel. Guard partial edge blocks.

Inverse gather copies RGB; out-of-bounds indices write black.

Disjoint outputs need no atomics/barrier. Work and storage: $O(W \times H)$.

Separate RGB buffers; $W/H > 0$; valid sizes, c/s. Uncompiled and unbenchmarked.

Host orchestration and completion

```
dim3 block(16, 16);
dim3 grid(1u + (unsigned(W)-1u)/block.x,
          1u + (unsigned(H)-1u)/block.y);
cudaMemcpy(d_in, h_in, bytes,
            cudaMemcpyHostToDevice);
rotate_rgb<<<grid, block>>>(
    d_in, d_out, W, H, c, s);
cudaGetLastError();
cudaMemcpy(h_out, d_out, bytes,
            cudaMemcpyDeviceToHost);
```

Illustrative excerpt, not a complete program: $W/H > 0$; $\text{bytes} = 3 \times W \times H$ checked in `size_t`; valid separate buffers; grid/device limits validated.

CPU loads/decodes, allocates host/device buffers, checks every API/launch/completion result, saves output and frees resources safely.

For this ordered synchronous-copy example, D2H supplies completed host data. A launch is asynchronous; launch-error checks alone do not establish successful completion.

Pinned host storage is page-locked and supports efficient DMA. Pageable memory may need runtime staging. Avoid excessive pinning.

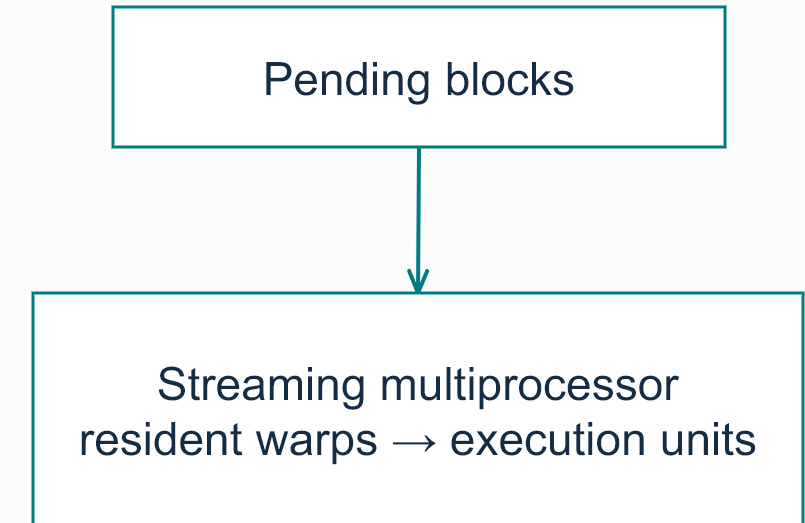
CUDA features that affect throughput

Example: independent brightness channels

```
size_t p = 3*(size_t(y)*W + x);
for (int ch=0; ch<3; ++ch) {
    int v = int(in[p+ch]) + delta;
    out[p+ch] = v<0 ? 0 : (v>255 ? 255 : v);
}
```

After coordinate/bounds checks; valid RGB buffers; $\text{delta} \in [-255, 255]$.

Resident warps hide latency; registers limit residency.
Coalescing groups nearby accesses into fewer transactions;
rotation can disrupt read locality.



SIMT = single instruction, multiple threads. A warp issues common instructions; divergent paths can reduce efficiency.

Concurrency helps only when memory access and resource use support it.

Launch shape and memory access

Choice	Mechanism	Performance implication
16 × 16 block	256 threads; each warp spans two 16-wide rows	Reasonable 2D starting point; row edges/alignment matter.
32 × 8 block	256 threads; each warp spans one 32-wide row	May improve row-oriented access; no universal winner.
Output gathering	Adjacent x threads write adjacent RGB pixels	Useful write locality; per-channel byte operations still require efficient transactions.
Rotated input reads	Angle changes the source addresses visited by a warp	Irregular gathers can reduce effective read bandwidth/cache reuse.
Higher residency	Registers, shared memory and block/thread limits constrain active warps	Helps hide latency; maximum occupancy is not automatically fastest.

Recompute grid for each block shape. Profile actual transactions, latency and registers before selecting a shape.

Expected speedup and its limits

Baseline: single CPU thread, same rotation and sampling

Also compare a multithreaded CPU; identical output and timing boundaries.

$$\text{Speedup} = T_{\text{CPU}} / T_{\text{GPU,total}}$$

$$T_{\text{GPU,total}} = \text{allocation/setup} + H2D + \text{kernel} + D2H$$

Many GPU execution units and high device bandwidth favor parallel throughput. Large or resident images amortize setup; gathered reads may limit bandwidth.

Amdahl: $S = 1 / (s + (1-s)/a)$. s is the unchanged serial share; a accelerates the remainder. $s = 0.20$ gives a 5× ideal ceiling before added transfers. Theoretical, not measured.

Expect gains for large or resident workloads; measure the whole pipeline.

Streams, overlap and completed output

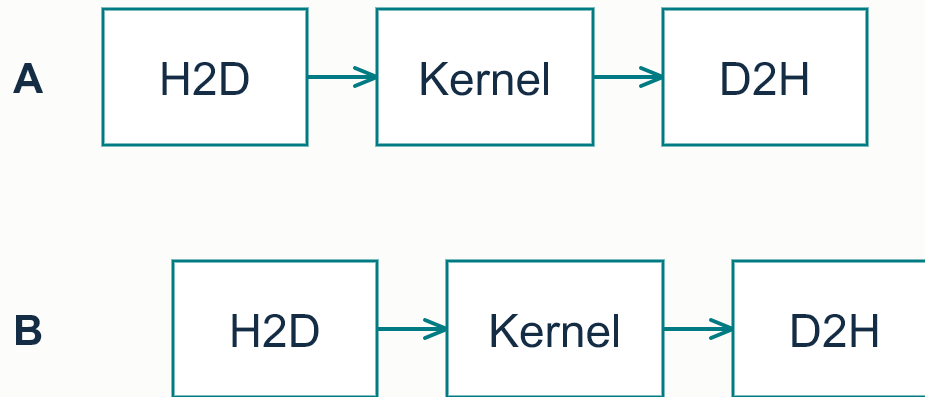
Ordered work in one stream

```
cudaMemcpyAsync(d_in, h_in, bytes,  
    cudaMemcpyHostToDevice, stream);  
rotate_rgb<<<grid, block, 0, stream>>>(  
    d_in, d_out, W, H, c, s);  
cudaMemcpyAsync(h_out, d_out, bytes,  
    cudaMemcpyDeviceToHost, stream);  
// After enqueueing all batch images:  
cudaStreamSynchronize(stream);
```

Batch: enqueue all streams before waiting. Check every API, launch and completion result; submission is not completion.

Separate pinned/device buffers and non-default streams; synchronize before reuse. More buffers cost memory. Keep images on the GPU across operations to avoid repeated transfers.

Independent images / streams



Possible overlap depends on supporting hardware and available resources.

Overlap can shorten the batch critical path; it changes scheduling, not total work.

Conclusions and next measurements

GPU rotation

Inverse gathering exposes independent pixel work. SM scheduling, memory locality and transfer costs determine achieved performance.

Choose a CPU baseline and compare completed equivalent pipelines. GDS is conditional on storage bottlenecks, a compatible data path and verified support.

Next: compile the teaching examples, check output against a CPU reference and profile representative image sizes/angles/batches. No result is assumed.

Responsible scaling

More compute can enable valuable research while concentrating access and increasing resource demand. Approve scale for useful outcomes, with safeguards.

Combine job-level energy/resource evidence, separate security/fairness evaluation, transparent allocations and deadline-aware scheduling.

Next: measure a real cluster job's energy, quality, scaling efficiency and allocation outcomes; revise policy when evidence or conditions change.

Optional supporting material

Task 1: implementation expansions

The Task 1 answers already contain the rotation formula, worked calculation, kernel, host launch, launch-shape comparison and stream/overlap example.

The following slides expand coordinate conventions and complete source examples, then give an optional profiling plan and GDS background.

Task 2: supplementary comparisons

The Task 2 answers already contain energy/carbon calculations, research comparisons, governance and fairness decisions, access policy and accountable scheduling.

Additional parallelism and cross-topic comparison tables follow. Website references and one brief AI-use declaration complete the deck.

Rotation coordinates and sampling

Coordinate convention

Input/output: contiguous 8-bit RGB; positive width W and height H ; x points right and y points down. Input and output buffers are distinct.

Rotate around $cx = (W - 1)/2$ and $cy = (H - 1)/2$. Positive θ is visually counterclockwise; the host supplies $c = \cos \theta$ and $s = \sin \theta$ in radians.

Forward image-coordinate rotation uses $[c \ s; -s \ c]$. Inverting it gives source offsets $[c \ -s; s \ c] \times$ output offsets, then restores the center.

Deliberate design choices

Nearest-neighbour indices: $ix = \text{floor}(sx + 0.5)$, $iy = \text{floor}(sy + 0.5)$. Copy three channels if the rounded indices are in bounds; otherwise write black.

The fixed $W \times H$ canvas can crop rotated corners. Nearest-neighbour sampling can alias; bilinear interpolation improves smoothness but adds source reads/arithmetic.

Forward scatter can leave holes or competing writes after rounding. Inverse gathering assigns every output exactly one writer. NPP offers library rotation with explicit interpolation/ROI choices.

Crafted CUDA example: RGB brightness

```
__global__ void brightness_rgb(const unsigned char* input,
                               unsigned char* output,
                               int width,
                               int height,
                               int brightness) {
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }
    const size_t offset =
        (size_t(y) * size_t(width) + size_t(x)) * 3;
    for (int channel = 0; channel < 3; ++channel) {
        const int value = int(input[offset + channel]) + brightness;
        output[offset + channel] =
            value < 0 ? 0 : (value > 255 ? 255 : value);
    }
}
```

Same operation on independent pixels: a data-parallel kernel.

Assume brightness in $[-255, 255]$. Integer addition avoids uint8 wrap; clamp each channel to $[0, 255]$.

Adjacent threads touch adjacent pixel locations, providing useful locality.

Illustrative CUDA C++; uncompiled and unbenchmarked.

Crafted CUDA example: inverse mapping

```
__global__ void rotate_rgb(const unsigned char* input,
                          unsigned char* output,
                          int width,
                          int height,
                          float c,
                          float s) {
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }

    const float cx = (width - 1) * 0.5f;
    const float cy = (height - 1) * 0.5f;
    const float sx = c * (x - cx) - s * (y - cy) + cx;
    const float sy = s * (x - cx) + c * (y - cy) + cy;
    const int ix = int(floorf(sx + 0.5f));
    const int iy = int(floorf(sy + 0.5f));
```

Global coordinates come from block/thread indices.

The guard handles partial edge blocks before any memory access.

The host computes sine/cosine once per angle.

Nearest-neighbour mapping uses the documented y-down convention.

Crafted CUDA example: safe output writes

```
const size_t destination =
    (size_t(y) * size_t(width) + size_t(x)) * 3;

if (ix >= 0 && ix < width && iy >= 0 && iy < height) {
    const size_t source =
        (size_t(iy) * size_t(width) + size_t(ix)) * 3;
    for (int channel = 0; channel < 3; ++channel) {
        output[destination + channel] = input[source + channel];
    }
} else {
    for (int channel = 0; channel < 3; ++channel) {
        output[destination + channel] = 0;
    }
}
}
```

RGB address = $3 \times (\text{row} \times \text{width} + \text{column})$.

Every valid output gets three channels, including black borders.

No atomics, mutex or block barrier is needed: writers have disjoint destinations.

Total work $\Theta(W \times H)$;
input/output storage $\Theta(W \times H)$ each.

Asynchronous work within one stream

```
cudaError_t enqueue_rotation(const unsigned char* h_input,
                             unsigned char* h_output,
                             unsigned char* d_input,
                             unsigned char* d_output,
                             size_t bytes, int width, int height,
                             float c, float s,
                             cudaStream_t stream) {
    const dim3 block(32, 8); // 256 threads.
    const dim3 grid = rotation_grid(width, height, block);
    cudaError_t status = cudaMemcpyAsync(d_input, h_input, bytes,
                                         cudaMemcpyHostToDevice, stream);

    if (status != cudaSuccess) return status;
    rotate_rgb<<<grid, block, 0, stream>>>(d_input, d_output,
                                             width, height, c, s);

    status = cudaGetLastError();
    if (status != cudaSuccess) return status;
    return cudaMemcpyAsync(h_output, d_output, bytes,
                           cudaMemcpyDeviceToHost, stream);
}
```

Full enqueue function; rotation_grid uses the ceil-division shown in the host orchestration answer.

Pinned host buffers and checked sizes are preconditions.

Success means submitted work; wait before reading, reusing or freeing buffers.

On error, previously enqueued work may still be pending.

Fair speedup comparison and profiling

Comparable baselines

Primary baseline: one CPU thread performs the same RGB inverse mapping, nearest-neighbour rule and fixed canvas. Also compare a multithreaded CPU for a stronger practical baseline.

Use identical inputs/angles and verify identical or justified numerically equivalent outputs. Include file decoding/I/O in both paths or exclude it from both.

For the stated decoded-buffer comparison, include GPU allocation, H2D, launch, completed kernel and D2H; CPU timing covers the corresponding output allocation and rotation.

Measure the bottleneck

Report cold-start and warmed repeated runs separately. Use CUDA events for kernel elapsed time after completion and a host timer through final completion for total latency.

Vary image size, angle, block shape and batch size; record hardware, transfer mode, register use and effective memory/link bandwidth.

The sequential sum does not apply unchanged to overlapping streams: measure the completed batch critical path. Lower kernel time alone cannot establish end-to-end speedup.

GDS decision for the proposed pipeline

Conventional decoded-image path

CPU decoding requires host-visible compressed bytes and produces pixels in RAM. Those pixels are copied to the GPU; the rotation kernel then runs.

A single small image already in RAM gains little from bypassing a storage step that is no longer on its path.

GDS does not decode an image, bypass an application's CPU transformation, or make GPU memory access itself faster.

Conditional batch alternative

Large storage-bound batches of GPU-ready pixels, or a compatible GPU decoding pipeline, may avoid host staging via cuFile and storage-side DMA.

Verify the GPU, storage/filesystem, software and PCIe topology support the direct path. Compatibility fallback uses host staging and is not evidence of direct DMA.

Adopt only if completed batch time/CPU overhead improve on the real workload; include extra buffers, decoding and storage bandwidth. Otherwise retain conventional copies.

References

[1] NVIDIA, “Programming model,” CUDA Programming Guide, online documentation, accessed Oct. 8, 2026.

<https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html>

[2] NVIDIA, CUDA C++ Best Practices Guide, online documentation, accessed Oct. 8, 2026.

<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html>

[3] NVIDIA, GPUDirect Storage Overview Guide, online documentation, accessed Oct. 8, 2026.

<https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html>

[4] NVIDIA, GPUDirect Storage Design Guide, online documentation, accessed Oct. 8, 2026.

<https://docs.nvidia.com/gpudirect-storage/design-guide/index.html>

References

[5] NVIDIA, “Parallelism strategies guide,” Megatron Core, online documentation, accessed Oct. 8, 2026.

<https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/parallelism-guide.html>

[6] International Energy Agency, Key Questions on Energy and AI, executive summary, Apr. 16, 2026, CC BY 4.0, accessed Oct. 8, 2026.

<https://www.iea.org/reports/key-questions-on-energy-and-ai/executive-summary>

[7] U.S. National Science Foundation, “NAIRR at 2 years: Advancing American artificial intelligence innovation and leadership,” Mar. 19, 2026, accessed Oct. 8, 2026.

<https://www.nsf.gov/cise/updates/nairr-2-years-advancing-american-artificial-intelligence>

[8] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in Proc. ACL, 2019, pp. 3645–3650, doi: 10.18653/v1/P19-1355.

<https://aclanthology.org/P19-1355/>

References

[9] P. Li, J. Yang, M. A. Islam, and S. Ren, “Making AI less ‘thirsty’: Uncovering and addressing the secret water footprint of AI models,” arXiv:2304.03271v5, Mar. 26, 2025; first submitted 2023.

<https://arxiv.org/abs/2304.03271v5>

[10] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” arXiv:1907.10597, 2019.

<https://arxiv.org/abs/1907.10597>

[11] L. Zhu, Z. Liu, and S. Han, “Deep leakage from gradients,” arXiv:1906.08935v2, 2019.

<https://arxiv.org/abs/1906.08935v2>

[12] I. O. Gallegos et al., “Bias and fairness in large language models: A survey,” Computational Linguistics, vol. 50, no. 3, pp. 1097–1179, 2024, doi: 10.1162/coli_a_00524.

<https://aclanthology.org/2024.cl-3.8/>

References

[13] N. Ahmed and M. Wahed, “The de-democratization of AI: Deep learning and the compute divide in artificial intelligence research,” arXiv:2010.15581v1, 2020.

<https://arxiv.org/abs/2010.15581v1>

[14] A. Radovanovic et al., “Carbon-aware computing for datacenters,” arXiv:2106.11750v1, 2021.

<https://arxiv.org/abs/2106.11750v1>

[15] OECD, “OECD AI Principles,” adopted 2019, updated 2024, accessed Oct. 8, 2026.

<https://www.oecd.org/en/topics/ai-principles.html>

[16] NIST, AI Risk Management Framework 1.0 Playbook, framework 2023, current online resource, accessed Oct. 8, 2026.

<https://airc.nist.gov/airmf-resources/playbook/>

[17] NVIDIA, “Rotate,” NPP Image Geometry Transforms, CUDA 13.0.3 archive documentation, accessed Oct. 8, 2026.

https://docs.nvidia.com/cuda/archive/13.0.3/npp/image_geometry_transforms.html#rotate

AI use declaration

AI model and tool

OpenAI GPT-6, accessed through Codex.

How AI was used

Research and source checks; drafting and revision of explanations and CUDA teaching examples; creation of diagrams and charts; presentation formatting and assembly.